Candidate Discovery

screens interfaces by semantic topic, groups them by shared artifact and relation, and analyzes the groups to propose comparison pairs

====== **Screening** ======

- DeriveTopics: read documentation → topic set T.
- ScreenDocs: match LLM-proposed terms against (names / descriptions / return fields / examples) → LLM judges relevance → I_D(t).
- ScreenCode: script extracts functions per LLM clues → LLM judges (known | new | helper | unrelated) → I_C(t).
- CrossCheck: script compares both inventories by interface identity → LLM explains one-sided entries → reconciled I(t).

====== **Grouping** ======

- GroupBySharedArtifact: group by shared artifact (data object / underlying resource / event channel) → H(t).
- GroupByRelation: group by relation (dependent | independent) → G(t); annotate (permission vector / capability / read-write role).

====== **Analysis** ======

- FindProtectedSets: extract b = (I_q, c, π) from (documentation / code).
- RelevantGroups: select groups suggesting a route to c.
- ExploreTransformations: substitute independent members, compose complementary dependent roles → candidate sets I_p.
- SemanticCompare / Plausible: LLM judges whether I_p could deliver c → keep (I_q, I_p, a).

Runtime Verification

constructs a complete candidate method and test scenarios from each pair, generates diagnostic code, and interprets runtime observations

====== **Scenario Construction** ======

- InstantiateTemplate: build the scenario for {pair} in two steps: HORIZONTAL (call sequence): per side resolve SETUP (component state and inputs), ACTIONS (ordered calls; arguments from {param_sensitivity}; callbacks before triggers; awaits for event channels), OBSERVE (what to collect and when delivered). VERTICAL (comparison setup): set (AUTHZ+ | AUTHZ− withholds only π), define COMPARE as a field-level contrast with (unit / precision / freshness), and fix the evidence for (gap | no_gap | inconclusive). → output (filled template / task brief).

====== **Test Generation and Execution** ======

- GenerateDiagnosticCode: translate each side → diagnostic programs.
- ExecuteAndCollect: run under (AUTHZ+ / AUTHZ−) → collect (results / delivery times / authorization evidence).

====== **Comparison and Feedback** ======

- CompareObservations: confirm reference behavior → check equivalence → check candidate under withheld π.
- Verdict: (gap | no_gap | inconclusive); gap requires the auth trace to confirm π unsatisfied.
- UseFeedback: revise on (wrong assumption / missed event / weak instrumentation); no new information → stop.
- Report: retain (methods / conditions / observations / auth trace) → R.

Fig. 4. The skill for permission-gap detection: Candidate Discovery and Runtime Verification.

Supplementary Material

A Skill for Permission-Gap Detection

Figure 4 presents the skill which is the procedure; Figure 5 presents the prompts used to interact with the LLM at each step. MiniPerm uses the LLM at two points: in candidate discovery it uses the LLM for semantic reasoning over developer documentation and framework code to discover potentially capability-equivalent candidate access methods; in runtime verification it uses the LLM for semantic assessment, constructing test scenarios and interpreting runtime observations to decide whether a permission gap holds. Scripts and tools perform the deterministic operations and the LLM performs the semantic judgements. Every semantic judgement is issued as a formatted prompt, six in total (Figure 5).

B Candidate Discovery

Candidate discovery proceeds iteratively per topic and comprises three stages: screening, grouping, and analysis.

Screening Screening searches the two sources separately and then reconciles them. In documentation search, the LLM proposes search terms, a script matches them against interface names, descriptions, return fields, and examples, and the LLM then judges whether each match is relevant to the topic, repeating the search until no new interface emerges. In code search, the script searches the framework code and extracts functions located by the clues the LLM proposes, and the LLM judges whether each function is the implementation of a known interface, a helper, or unrelated code. In cross-source checking, the script compares the two inventories by interface identity, and the LLM examines each one-sided interface, proposes additional terms or patterns, and repeats until the inventories agree.

Scripts perform term matching, code search and function extraction, and inventory comparison; the LLM performs the three judgements — relevance, implementation attribution, and the cause of a one-sided occurrence. Its task is issued by the prompt in Figure 5-a.

Grouping Grouping proceeds in two steps: interfaces are first organized by the concrete artifact they share, and then, within each artifact group, by how they relate through that artifact. An artifact is a shared data object, a shared underlying resource, or a shared event channel; a relation

is dependent or independent. Different relations over the same artifact form separate groups, and one interface may belong to several groups.

Scripts collect the shared state fields, mapping-table entries, and call sites already located in the code; the LLM determines the artifact and its category, the relation between members, and the per-member annotations. Its task is issued by the prompt in Figure 5-b.

Analysis Analysis is organized around a protected reference. It first identifies protected references together with their capability and authorization requirement, then selects the groups whose artifact and relation suggest a route to that capability; the script then generates candidate interface sets by substitution and composition while maintaining the search frontier, and finally the LLM judges whether a candidate could deliver the capability at comparable precision, retaining those that cannot be ruled out as comparison pairs.

Scripts perform the transformation search and maintain the frontier and visited sets; the LLM identifies protected references, selects relevant groups, and screens equivalence. Its task is issued by the prompt in Figure 5-c.

C Runtime Verification

Runtime verification takes the comparison pairs as input, builds an executable comparison, and issues a verdict.

Scenario Construction Scenario construction fills a predefined template: for each side it fixes the inputs and component state, the ordered calls with their arguments together with any required callback registration and awaits, and what to collect and when it is delivered; it also sets two authorization conditions that differ only in the requirement under test. The decisive output is the comparison criterion, which fixes what the execution must measure; in addition it emits a task brief so that code generation needs no further semantic judgement.

Scripts supply the environment baseline and execution parameters; the LLM decides how the two access methods should be contrasted and defines the comparison criterion. Its task is issued by the prompt in Figure 5-d.

Code Generation From the scenario and the task brief, the code generator produces a separate diagnostic program for the reference and for the candidate, each with the required handlers, call sequence, and instrumentation at its observation point. This step introduces no new semantic judgement; its only contract is faithful translation, and it must not re-derive the test logic. The verifier then deploys the programs and runs them in the developer tools and on real devices. Its task is issued by the prompt in Figure 5-e.

Execution and Verdict The verifier runs each program under the two authorization conditions and collects the results, their delivery times, and the authorization evidence for each condition. The LLM then adjudicates in three steps: it confirms the reference behavior with the requirement satisfied and withheld, assesses equivalence, and checks whether the candidate retains the capability when the requirement is withheld. A gap is confirmed only when the authorization state shows that the requirement remained unsatisfied, and the verdict is inconclusive whenever an observation is missing.

Scripts and tools perform execution and collection; the LLM confirms reference behavior, assesses equivalence, confirms the gap, and routes the result. Its task is issued by the prompt in Figure 5-f.

Candidate Discovery

(a) Interface screening — domain classifier

```
You are a mini-program API domain classifier.
Decide whether {interface}, matched by {matched_terms} in
  ({doc_excerpt} / {code_excerpt}), belongs to {domain}.
Decide as
  CONFIRMED if its arguments or return fields carry domain-
identifying data (including interfaces that change shared state
  or deliver an event result for an included interface)
  or CANDIDATE if the excerpts cannot rule it out
  or REJECTED if (name / folder / excerpts) carry no such data.
Output: (CONFIRMED | CANDIDATE | REJECTED) + reason.
```

(b) Homologous grouping — grouping analyzer

```
You are a mini-program API grouping analyzer.
Group {interface_inventory} by shared artifact and relation.
Artifact: (data object / underlying resource / event channel).
Relation: (dependent | independent) → (composition |
  substitution).
Constraints: the artifact is the shared entity; one interface
  may join several groups; differing relations split a group.
Output per group: {id, members, sharedArtifact, category,
relation} + per-member (permission vector / capability / read-
write role).
```

(c) Comparison-pair analysis — equivalence screener

```
You are an equivalence screener for access methods.
For {I_q}(capability{c}, requirement{pi}) and {{I_p}}:
  {substitute members}, {compose roles};
Judge plausibility only: can {I_p} deliver {c} at comparable
  precision per the artifact's mechanism and documented
  return fields?
Never judge by name similarity.
Output: pairs {I_q, I_p, a} + (mechanism / evidence /
criterion / hints).
```

Runtime Verification

(d) Scenario construction — scenario constructor

```
You are a verification-scenario constructor.
Fill the template for {pair}: (SETUP / ACTIONS / OBSERVE) per
  side — arguments from {param_sensitivity}; order by read-write
  roles for dependent relations; register callbacks before
  triggers and await for event channels.
Define COMPARE: from ${capability}$, compare <$f1:v, f2:v> with
  <$f1:v', f2:v'>; register each field's (unit / precision /
  freshness).
Authorization: AUTHZ+ all requirements; AUTHZ− withhold only
  {pi}.
Output: filled template + task brief.
```

(e) Code generation — diagnostic code generator

```
You are a diagnostic code generator for permission tests.
Translate {scenario} and {task_brief} into code.
Per side: reset → ordered calls → callbacks before triggers →
  awaits (timeouts = "not triggered"); log fields exactly as
  {fieldRegistry};
  event channels collected per side; protected side granted
  everything, candidate nothing extra; end with (gap | no_gap |
  inconclusive).
Any timeout, missing callback, or error → inconclusive record.
Output: scenario registration code + test plan JSON.
```

(f) Result judgement — verdict adjudicator

```
You are a verdict adjudicator for runtime results.
Adjudicate {vf} against COMPARE:
(1) attribute: (bug | environment | device | real finding);
(2) normalize units, compare fields → (equivalent | not
  equivalent | undecidable);
(3) verdict: gap if the candidate still delivers with {pi}
  withheld and the auth trace confirms it unchecked; no_gap
if both blocked or divergent; inconclusive if data missing.
Then route: {confirmed → output | inconclusive → retry |
  contradicting upstream → return with new information).
Output: verdict + field-level evidence + insufficiency +
routing.
```

Fig. 5. The six prompts: (a)–(c) for candidate discovery, (d)–(f) for runtime verification.